

CSC 4553

Operating Systems

This class

This class

- Too much to cover in one term: survey of important topics
- Course Canvas: <https://canvas.calpoly.edu/courses/187882>
- Syllabus: <https://schmittpaul.github.io/CSC4553F26/syllabus.pdf>
- Waitlists...
- What OSes do you all run?

Who am I?

- <https://pschmitt.net>
- Academics
 - PhD at UCSB
 - Researcher at UC Berkeley (ICSI)
 - Some other stops... Princeton, USC, Stanford, UH
- Privacy startup co-founder & CEO, measurement startup COO

Who are you?

- Name / Major / Year
- Where do you identify as your hometown?

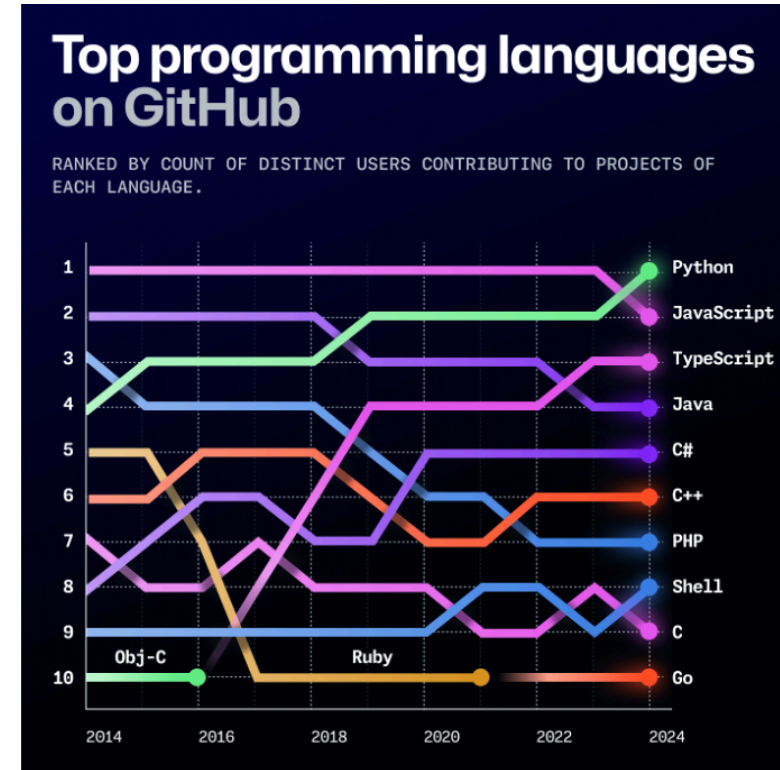
How to succeed in this class

- Turn things in
 - Labs are due on Fridays
 - Quizzes are due on Mondays
 - Programming assignments are due on Wednesdays
- Attend class
 - ... on time. Lectures will start on time
- Ask questions
 - I'm very flexible about how much we cover this term
 - I would rather teach less and have everyone understand it
 - Our back-and-forth during class is the one of the few indicators I have of how much you are absorbing
- Talk to me if you are struggling

What are we doing here?

- How many of you have participated in OS development?
- How many of you regularly program in languages that use operating system abstractions directly?
- And C is a decreasingly popular language!
- So why study operating systems?
Why is this class even offered?

Why is it required?



Why take this class

- You are required to do so in order to graduate
- **Reality**: this is how computers really work, and as a computer scientist or engineer you should know how computers really work
- **Ubiquity**: operating systems are everywhere and you are likely to eventually encounter them or their limitations
- **Beauty**: operating systems are examples of mature solutions to difficult design and engineering problems. Studying them will improve your ability to design and implement abstractions

Course progression

- Introduction to operating system abstractions and structures
- Abstracting and multiplexing:
 - **CPU**: interrupts, context, threads, processes, processor scheduling, thread synchronization, deadlocks
 - **Memory**: memory layout, address translation, paging and segmentation, address spaces, translation caching, page fault handling, page eviction, swapping
 - **File systems and storage**: disk scheduling, on-disk layout, files, buffer cache, crash and recovery
 - **Security** (time permitting)

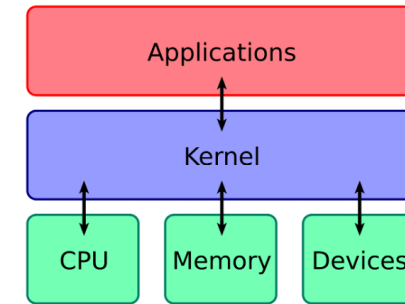
Background

Questions to consider

- What is an operating system?
- What do they do?
- How did OSes evolve?
- How do the requirements differ for different OSes?

What is an operating system?

- Operating System:
 - A **computer program** that
 - **multiplexes** hardware resources and
 - implements useful **abstractions**.
- The OS is just another computer program. Has the highest privilege. The core of the OS consists of the **kernel**
- **Multiplexing** allows multiple people or programs to use the same set of hardware resources - processors, memory, disks, network connection - safely and efficiently.
- **Abstractions** simplify the usage of hardware resources by organizing information or implementing new capabilities.



OS history - how did we get here?

- Started out as **libraries** to provide common functionality across programs
 - See any issues with this approach?
- Later, evolved from **procedure call** to **system call**
 - What's the difference?
- Evolved from running a single program to **multiple processes concurrently**
 - New issues to solve?



Types of operating systems

- Desktop
- Time share/Mainframe
- Mobile
- Web
- Real-time
- Embedded
- Virtual Machines

Tl;dr

OSes continue to evolve as environments (i.e., constraints) change

What isn't clear?

Comments? Thoughts?

OS abstractions

Questions to consider

- What are the main abstractions OSes provide?
- What are the abstraction challenges?

Abstractions

- Abstractions *simplify application design* by:
 - hiding undesirable properties,
 - adding new capabilities, and
 - organizing information
- Abstractions provide an **interface** to application programmers that separates **policy**—what the interface commits to accomplishing—from **mechanism**—how the interface is implemented.

What are the abstractions?

- CPUs
 - Processes, threads
- Memory
 - Address space
- Disk
 - Files

Example OS abstraction: file systems

- What **undesirable properties** do file systems hide?
 - Disks are slow!
 - Chunks of storage are actually distributed all over the disk
 - Disk storage may fail!
- What **new capabilities** do files add?
 - Growth and shrinking
 - Organization into directories, searchability
- What **information** do files help organize?
 - Ownership and permissions
 - Access time, modification time, type, etc.

Abstraction tradeoffs - discussion

- Identify undesirable properties hidden by, new capabilities added, and info organization provided with these abstractions:
 - Process / threads
 - Address space

Abstraction pros / cons

- **Advantages** of OS providing abstractions?
 - Allow applications to reuse common facilities
 - Make different devices look the same
 - Provide higher-level or more useful functionality
- **Challenges?**
 - What are the correct abstractions?
 - How much should be exposed?

OS design requirements - what do we need?

- **Convenience**, abstraction of hardware resources for user programs
- **Efficiency** of usage of CPU, memory, etc.
- **Isolation** between multiple processes
- **Fairness** of access to contended resources
- **Reliability**, the OS must not fail
- Other:
 - **Security**
 - **Mobility**

Another view of the same list: the OS as a **resource manager** that shares resources “well.”

Challenges? What are the correct **mechanisms**? What are the correct **policies**?

What isn't clear?

Comments? Thoughts?

Boot chain

Questions to consider

- What is the chain of events that takes place before you are able to run a process?
- Where do the different parts of the chain reside? How are they called?

Boot is like a relay race

- A sequence of programs
- Each stage:
 - Has slightly more power
 - Loads the next stage
 - Jumps to it

Power-On

- CPU resets
- Instruction pointer set to a fixed address
- Execution begins in firmware (ROM)
- **The CPU does not know what an OS is**

Firmware (BIOS / UEFI)

- Initializes RAM
- Initializes minimal hardware
- Finds one program to run (bootloader)
- Loads it into memory
- Jumps to it

Bootloader's job (GRUB Example)

- Understand filesystems
- Load the kernel image
- Load the initramfs
- Jump to the kernel entry point
- Provides kernel command line args

```
1 root=/dev/sda1  
2 init=/sbin/init  
3 console=ttyS0
```

What the bootloader does **not** do

- Does not schedule processes
- Does not manage memory long-term
- Does not run the OS

Kernel

Think of the kernel in phases

1. Self-setup
2. Hardware abstraction
3. Userspace handoff

Kernel self-setup

- Sets up page tables
- Enables virtual memory
- Initializes scheduler
- Sets up interrupts

The kernel cannot rely on **anything** yet: it is building the world.

Kernel hardware abstractions

This is where hardware becomes an abstraction.

- Detects hardware
- Loads drivers
- Turns devices into files:
 - `/dev/sda`
 - `/dev/tty`
- Creates kernel threads

Userspace handoff

- Mounts the root filesystem
- Chooses one program to run
- Executes it with `execve()`

```
1 execve("/sbin/init", ...);
```

Init (PID 1)

- First userspace process
- Always PID 1
- Parent of all other processes

If PID 1 exits?

- The kernel **panics** or shuts down

What isn't clear?

Comments? Thoughts?

Kernel basics

Questions to consider

- What are the different kernel paradigms?
- Why would you choose one over the other?

Paradigms

- The OS offers a number of services. What should go in the kernel?
 - IPC
 - VFS
 - File system
 - Scheduler
 - Virtual Memory
 - Device drivers

Monolithic kernels

- Oldest, very common design (Linux, Windows 9x, BSDs)
- Single piece of code in memory
- Limited **information hiding**
 - One part of the kernel can directly access data and functions of other parts

Monolithic kernels (cont'd)

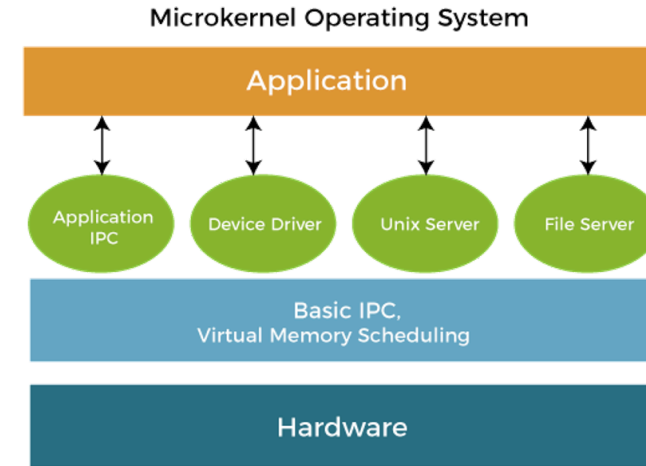
- Q: What happens when you need something new supported (new hardware device, new system call, new filesystem)?
- **Modules** (loadable kernel modules - LKMs) allow for flexibility, customization, support
 - Pros?
 - Memory savings
 - Flexibility
 - Minimal downtime
 - Cons?
 - (minor) fragmentation: the base kernel can be loaded contiguously in memory
 - Security (<https://github.com/m0nad/Diamorphine> **DO NOT USE THIS, IT'S SIMPLY TO SHOW YOU**)

Layered kernels

- Dijkstra created the THE OS in the 60s, introducing the concept Each inner layer is more privileged; required a trap to move down layers
- Hardware-enforcement possible
- Intel **announced** in May 2024 that the new architectures will only have rings 0 and 3

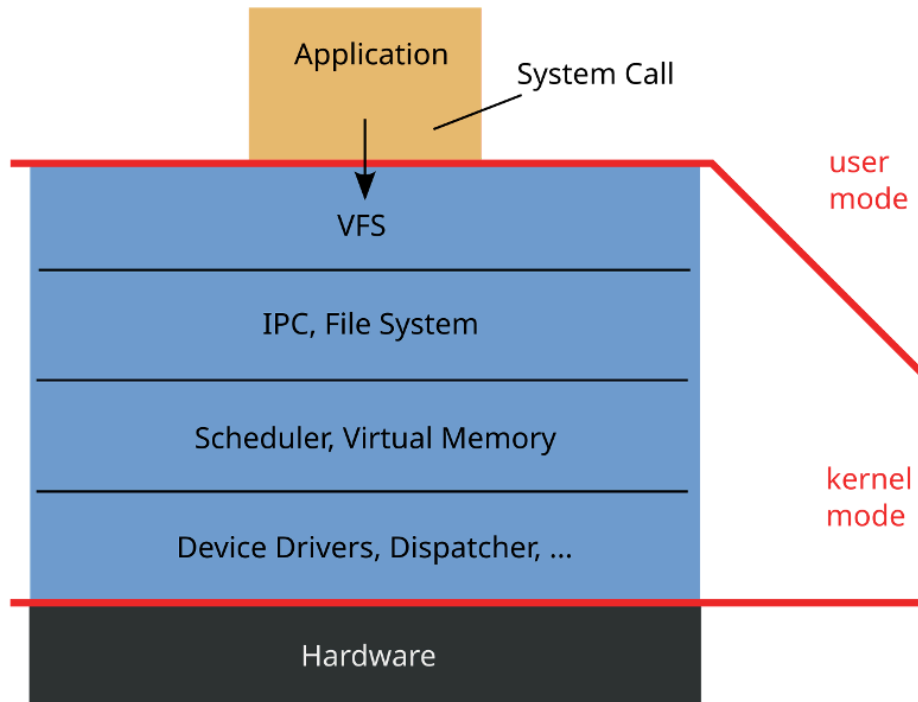
Microkernels

- Popular research area long ago, didn't win (although people remain interested in the principles)
- All non-essential components removed from the kernel, for modularization, extensibility, isolation, security, and ease of management
- A collection of OS services running in user space
- Downsides?
 - **Heavy communication costs** through message passing (marshaled through the kernel)

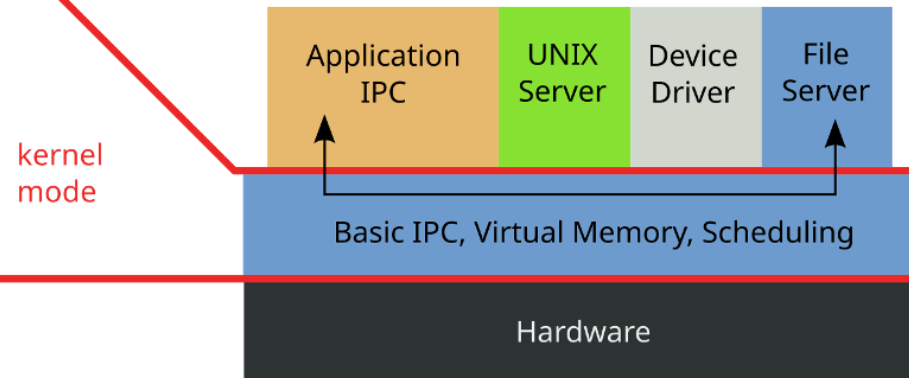


Monolithic vs Microkernel

Monolithic Kernel based Operating System



Microkernel based Operating System



Hybrid kernels

- Combine a monolithic core with microkernel ideas
- Small kernel handles critical services (scheduling, IPC, low-level memory)
- Other services (file systems, drivers) can run in kernel *or* user space
- Examples: macOS (XNU), Windows NT
 - XNU: Mach microkernel + BSD monolithic layer
 - Windows NT: microkernel-inspired, but most services run in kernel mode for performance
- Pragmatic compromise: **microkernel isolation where it matters, monolithic speed where it doesn't**

What isn't clear?

Comments? Thoughts?